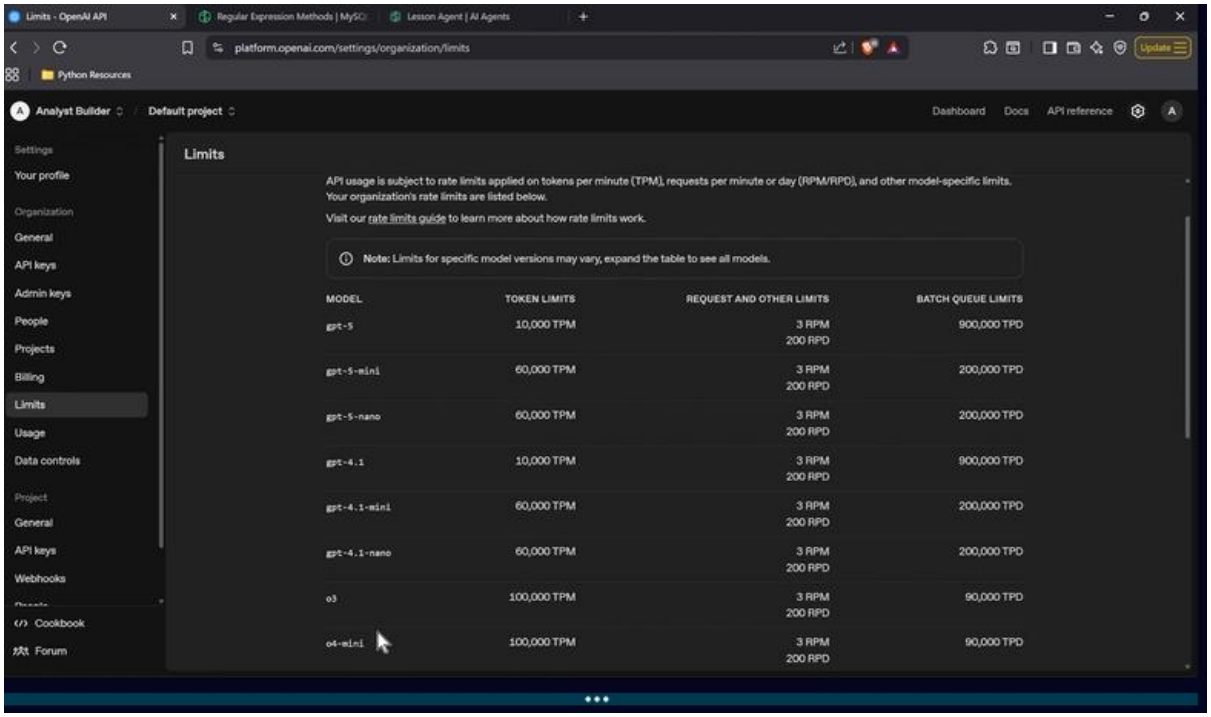


Limits - OpenAI API
Regular Expression Methods | MySQ
Lesson Agent | AI Agents
platform.openai.com/settings/organization/limits
Update
Python Resources
Analyst Builder
Default project
Dashboard
Docs
API reference
Settings
Your profile
Organization
General
API keys
Admin keys
People
Projects
Billing
Limits
Usage
Data controls
Project
General
API keys
Webhooks
Cookbook
Forum
Limits
API usage is subject to rate limits applied on tokens per minute (TPM), requests per minute or day (RPM/RPD), and other model-specific limits.
Your organization's rate limits are listed below.
Visit our rate limits guide to learn more about how rate limits work.
Note: Limits for specific model versions may vary, expand the table to see all models.
MODEL
TOKEN LIMITS
REQUEST AND OTHER LIMITS
BATCH QUEUE LIMITS
gpt-5
10,000 TPM
3 RPM
200 RPD
900,000 TPD
gpt-5-mini
60,000 TPM
3 RPM
200 RPD
200,000 TPD
gpt-5-nano
60,000 TPM
3 RPM
200 RPD
200,000 TPD
gpt-4.1
10,000 TPM
3 RPM
200 RPD
900,000 TPD
gpt-4.1-mini
60,000 TPM
3 RPM
200 RPD
200,000 TPD
gpt-4.1-nano
60,000 TPM
3 RPM
200 RPD
200,000 TPD
o3
100,000 TPM
3 RPM
200 RPD
90,000 TPD
o4-mini
100,000 TPM
3 RPM
200 RPD
90,000 TPD

WHAT IS AI?

Analyst Builder: AI for Data Professionals

What Is an LLM (Large Language Model)?

- Trained on **massive amounts of text** - books, websites, articles, and more.
- Learns **how words, phrases, and ideas connect** to form coherent language.

- Uses this knowledge to **generate text, answer questions, and hold conversations** naturally.
- Think of it as a system that's learned **patterns of human communication**, not just memorized facts.
- Foundation for tools like **ChatGPT, Claude, and Gemini** - all powered by LLMs.
- **Uses LLMs to make machines "smart"** - enabling them to learn, reason, and make decisions.
- **Generates diverse content** like text, images, music, and code.
- **Applied across industries** - powering chatbots, self-driving cars, medical diagnostics, and fraud detection.

TERMINOLOGY TO KNOW

Prompts

- A prompt is the text, question, or instruction you give to an AI.
- It guides how the AI responds and what kind of output it generates.
- Detailed prompts = better results - clarity, context, and specificity help the AI understand your intent.
- Think of it as giving directions: the clearer you are, the closer the AI gets to what you want.

Models

- A model is an AI program trained on data to recognize patterns and make predictions.
- It learns and improves over time as it processes more data and receives feedback.
- Companies build and update multiple models to enhance accuracy, speed, and capabilities.
- Think of it as the "brain" behind AI systems, powering how they analyze, respond, and adapt.

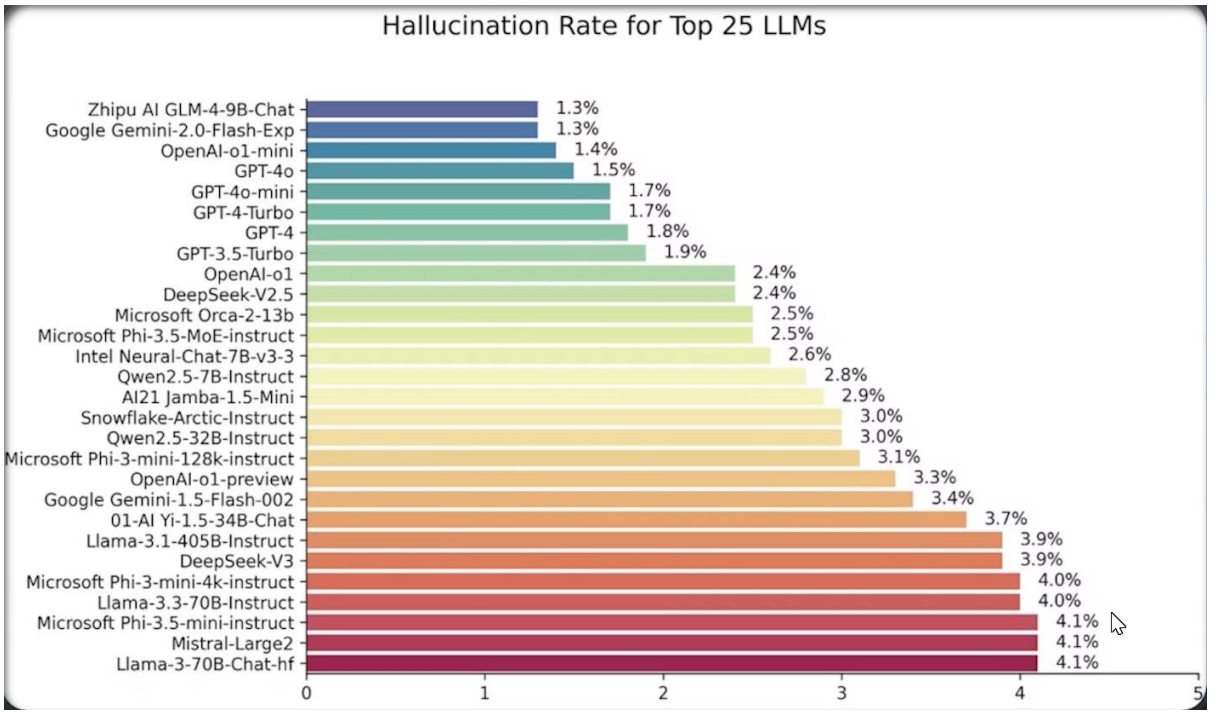
AI Agents

- An AI agent is a program that can take actions on its own to achieve a goal.
- It interacts with its environment - answering questions, controlling software, or even moving robots.

- Agents are designed to make decisions and adapt based on what they observe.
- Think of them as AI “doers” - systems that don’t just think, but act.

Hallucinations

- A hallucination happens when AI creates information that isn’t true or doesn’t exist.
- It occurs because the AI predicts patterns, not because it verifies facts.
- All AI systems experience this - it’s a limitation of how they generate language.
- Always treat AI outputs as suggestions to verify, not guaranteed truths.



AI USE CASES

- Healthcare: Diagnosing diseases, reading medical scans, and discovering new drugs.

- Finance: Fraud detection, credit scoring, and automated trading.
- Customer Service: Chatbots and virtual assistants that answer questions.
- Creativity & Productivity: Writing text, generating images, composing music, and coding.
- AI's reach continues to expand - transforming industries through automation and insight generation.

User Interfaces (ChatGPT, Gemini, Claude)

Links

ChatGPT: https://chatgpt.com/

- Can go up to 25MB for images

Gemini: https://gemini.google.com/

Claude: https://claude.ai/

Using AI Through a User Interface

- User interfaces (UIs) like ChatGPT, Gemini, and Claude make AI tools easy to use - no coding required.
- Type a question or task, and the AI generates a response instantly.

- Each tool has its own style and strengths - some are better for writing, others for data or coding.
- Most UIs include features like chat history, file uploads, and tone or format controls.
- Great for learning, brainstorming, automating tasks, and generating ideas quickly.

Using an API

Link to OpenAI API: https://platform.openai.com/docs/overview

Starter Code:

```
from openai import OpenAI



# 1) Put your API key here (or in your environment as OPENAI_API_KEY)

client = OpenAI(api_key="[put your key here]")



# --- DEMO 1: Ask a question directly (like ChatGPT) ---

print("------Demo 1: Asking any question-------")

question = "Are we alone in the universe?"

response = client.chat.completions.create(

    model="gpt-4o-mini",

    messages=[{"role": "user", "content": question}]
```

```
)

print("Q:", question)

print("A:", response.choices[0].message.content)

print()




## Another question -- What is the capital of France?
```

Using an API for AI

- An API (Application Programming Interface) lets developers connect AI tools directly into apps or workflows.
- Instead of chatting, you send data or prompts through code and receive structured responses back.
- APIs are used to automate AI tasks at scale - like summarizing documents, generating text, or analyzing data.
- Common examples include the OpenAI API, Gemini API, and Anthropic API (Claude).
- Gives you more control over inputs, outputs, and how AI fits into your system.

My code I used in my Databricks Notebook integrated with Genie:

```
from mlflow.deployments import get_deploy_client

# Initialize Databricks Foundation Model API client
client = get_deploy_client("databricks")

# --- DEMO 1: Ask a question directly (like ChatGPT) ---
print("------Demo 1: Asking any question-------")
question = "What is the capital of Australia?"
response = client.predict(
    endpoint="databricks-meta-llama-3.1-405b-instruct",
    inputs={
        "messages": [{"role": "user", "content": question}],
        "max_tokens": 500
    }
)
print("Q:", question)
print("A:", response['choices'][0]['message']['content'])
print()

# Another question -- Are we alone in this together?
question2 = "Are we alone in this together?"
response2 = client.predict(
    endpoint="databricks-meta-llama-3.1-405b-instruct",
    inputs={
        "messages": [{"role": "user", "content": question2}],
        "max_tokens": 500
    }
)
print("Q:", question2)
print("A:", response2['choices'][0]['message']['content'])
```

NOTE: Tokens are accounted for when we use rate limits (tokens per minute), requests per minute per day depending on the model, token limits, request and other limts and batch queue limits.

------Demo 1: Asking any question-------

Q: What is the capital of Australia?

A: The capital of Australia is Canberra.

Q: Are we alone in this together?

A: What a profound and philosophical question!

The phrase "alone in this together" is a bit of an oxymoron, as it suggests that we are simultaneously isolated and connected. However, I think it's a clever way to describe the human experience.

In many ways, we are alone in our individual experiences, thoughts, and emotions. Each person has their unique perspective, struggles, and triumphs. We may feel disconnected from others, even when we're surrounded by people.

On the other hand, we are also part of a larger collective, sharing this human experience with millions of others. We're all connected through our shared struggles, hopes, and desires. We're part of a global community, influenced

by each other's actions, and contributing to the world's complexities.

So, are we alone in this together? I'd say yes, in the sense that we're all navigating our individual journeys, but also, no, because we're all part of a larger, interconnected web of human experience.

This paradox is beautifully captured in the words of the poet Rainer Maria Rilke: "The only journey is the one within, and yet, we're all on this journey together."

What are your thoughts on this question? How do you experience the balance between individuality and connection with others?

MAIN TAKEAWAY

AI models work using **tokens**, which are small chunks of text (words, parts of words, punctuation, etc.). Every

prompt you send and every response the AI generates uses tokens, which act like the “fuel” for AI systems — similar to how petrol powers a car.

AI agents read instructions, process context, and generate outputs using these tokens, while systems apply limits such as **Tokens Per Minute (TPM)**, **Requests Per Minute (RPM)**, daily quotas, and **context window limits** (the amount of information the AI can “carry” or remember at once). Efficient prompt engineering focuses on getting better answers using fewer tokens by making prompts short, clear, and structured — like giving a driver precise GPS directions instead of vague instructions.

- **Token Example (Fuel Analogy):**
  - “Hello world!” ≈ 2–3 tokens
  - A 1,000-word document may use ~1,300 tokens
  - Analogy: Tokens are like fuel consumption — more text = more fuel used by the AI.
- **Efficient Prompt Example (Clear GPS Directions):**
  - Bad: “Please analyze every possible detail about marketing.”
  - Better: “Give 3 marketing strategies for SaaS startups.”

    - Analogy: A vague prompt is like saying “just drive somewhere,” while a clear prompt is like entering an exact destination into GPS.
- **Context Window Example (Vehicle Storage Capacity):**
    - If a model supports 128k tokens, it can only “remember” information that fits within that limit during one conversation.
    - Analogy: A car can only carry a limited amount of luggage/passengers before it becomes overloaded.
- **Rate Limit Example (Driving Limits):**
    - 60 RPM = only 60 API requests allowed per minute
    - 100k TPM = total prompt + response tokens allowed each minute
    - Analogy: RPM is like the number of trips a taxi can take per minute, while TPM is like fuel usage limits.
- **AI Agent Efficiency Techniques (Smart Driving):**
    - Use summaries instead of full documents
    - Retrieve only relevant information (RAG)
    - Limit outputs with instructions like “answer in 5 bullet points”

    - Analogy: A smart driver uses optimized routes instead of carrying every map or taking unnecessary detours.
- **Training Example (Teaching Drivers):**
    - Models are trained on billions/trillions of tokens from books, websites, code, and conversations to learn language patterns and reasoning.
    - Analogy: Training an AI is like teaching millions of drivers every road, traffic rule, shortcut, and driving pattern.
- **Batch Processing Example (Carpooling Efficiency):**
    - AI systems often process multiple requests together to improve performance and reduce cost.
    - Analogy: Similar to ridesharing or carpooling to save fuel and maximize efficiency.
- **Simple Analogy Summary:**
    - Tokens = fuel
    - Prompt = GPS directions
    - Context window = vehicle storage size
    - AI agent = self-driving chauffeur
    - Prompt engineering = giving efficient driving instructions

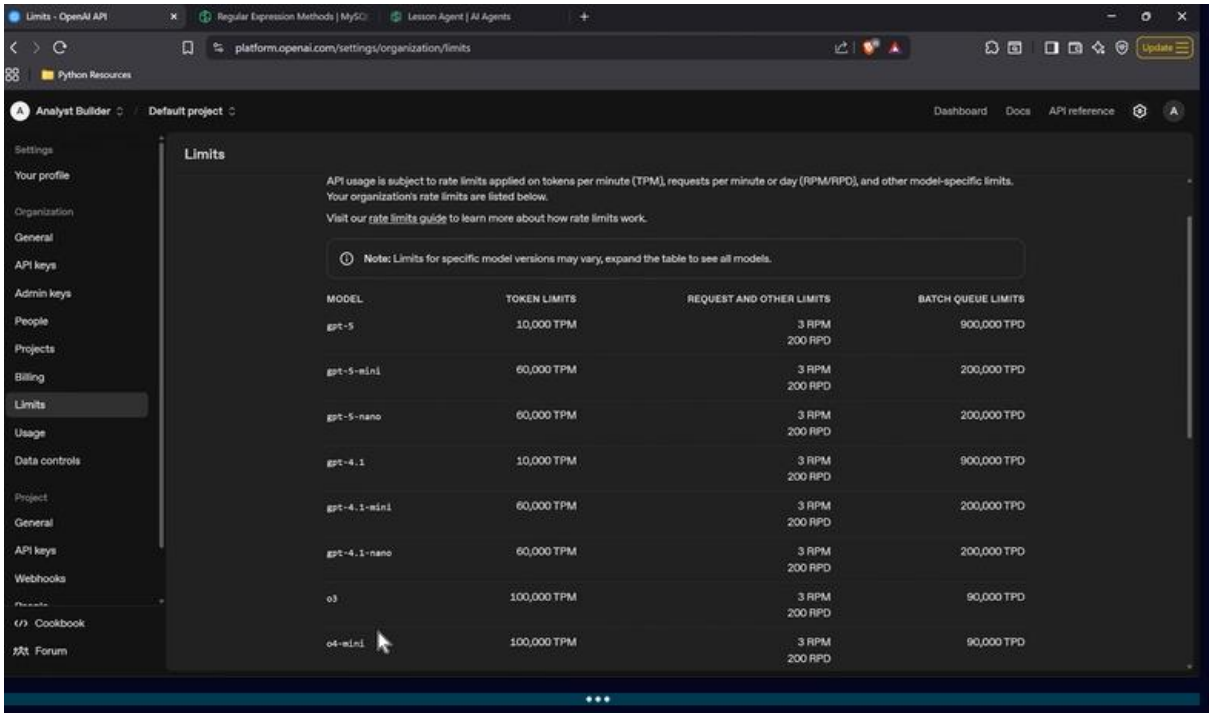
- RAG = retrieving only the necessary map section
- Batch processing = ridesharing efficiency

The main idea behind prompt engineering and AI efficiency is:

getting the best destination (output) using the least amount of fuel (tokens) while giving the clearest possible directions (prompts).

GOAL (main purpose why we use tokens):

The main takeaway is that AI models work by processing tokens, which act like fuel powering the system. Efficient prompt engineering means giving the AI clear, specific instructions so it can produce better results using fewer tokens, less memory, and less compute power. Just like a car with good GPS directions uses fuel more efficiently, well-structured prompts help AI agents work faster, cheaper, and more accurately.



Code Editors

Link to VSCode: https://code.visualstudio.com/

Using AI in Code Editors

- Tools like GitHub Copilot and Cursor bring AI directly into your coding environment.
- They suggest code, fix errors, and explain logic as you type - like having an assistant in your IDE.
- Works inside editors such as VS Code, JetBrains, and Visual Studio.
- Uses models trained on massive code datasets to predict your next line or generate full functions.
- Great for learning new languages, speeding up development, and reducing repetitive work.

- The more context (comments, function names, or docstrings) you give, the smarter and more accurate the suggestions become.

In create code:

- Create a pandas script to read in a csv file and summarize it.
- please create a new file called pandas.py that explains some of the basics of pandas and shows some basic code.

Fundamentals of Prompting

- A prompt is how you communicate with AI - it's your instruction, question, or request.
- Be clear and specific: The more detail you include, the better the output.
- Give context: Tell the AI what role to take (e.g., "act as a data analyst") or what format you want (e.g., "in bullet points").
- Iterate: Refine your prompt based on the response - prompting is a back-and-forth process.
- Structure helps: Use lists, examples, or constraints ("under 100 words," "use a friendly tone") to guide the AI.

- Great prompting turns AI into a powerful collaborator instead of just a responder.

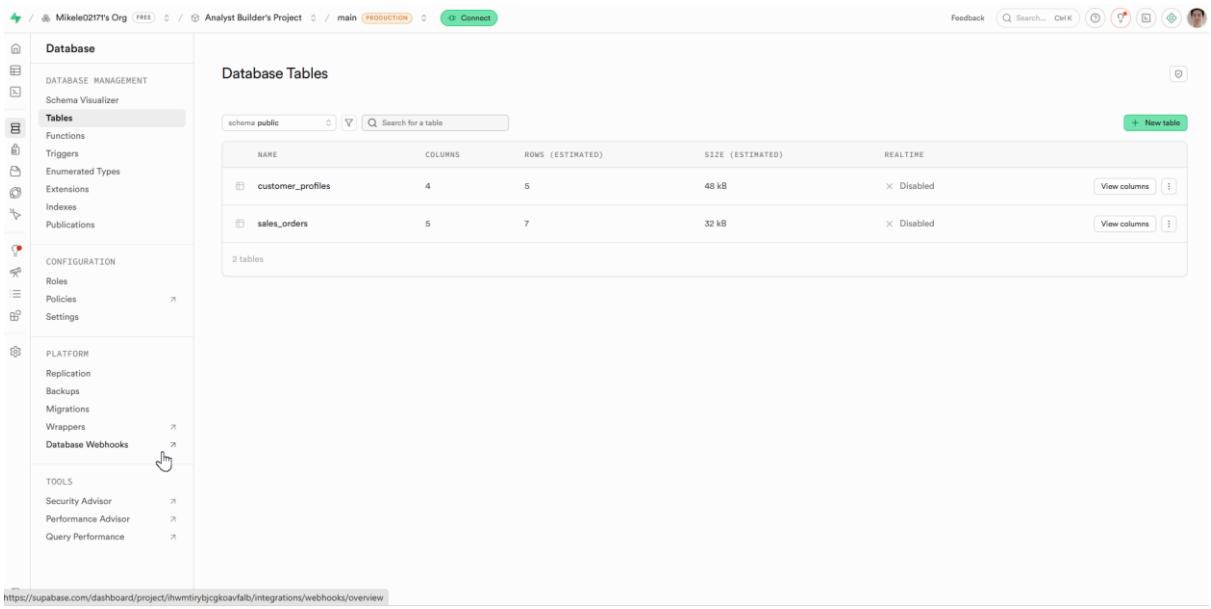
PROMPTS TO ASK CHAT-GPT

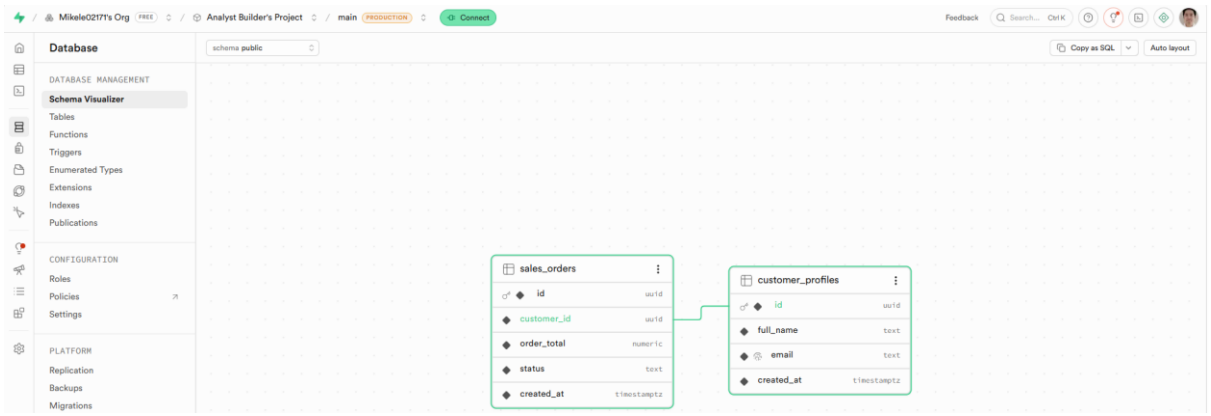
- I want you to give me portfolio project ideas for a data analyst in Melbourne, Victoria Australia
- I'm an aspiring data analyst located in Melbourne, Victoria Australia. I want to get into healthcare, retail, or sports and I want 5 projects THAT I CAN USE. Can you provide project ideas along with public datasets that I can use. Rank the projects from best to worst. I want a few projects on data cleaning, EDA, and Data Visualization. (MENTION FOCUS AREAS, DATASET WITH THE LINK AND DELIVERABLE EXAMPLE! TO PERFORM THIS).
- i WANT YOU TO GIVE ME PORTIFOLIO PROJECT IDEAS FOR A DATA ANALYST.
- 1, 2, 4 AND 5 LOOKS GOOD. COULD YOU GO MORE IN DEPTH ON THOSE WITHIN THE VICTORIAN, AUSTARLIAN JOB MARKET FOR AN ASPRIING DATA ANALYST.

AI In SQL

Link to Supabase: https://supabase.com/

NOTE: Writing these prompts into AI assistant may not match with the video, following exactly where you save your schema (we want to save it into public). Check the Schema Visualizer to ensure there is a relationship between these two tables. Otherwise ask the AI assistant to create it.





Use AI Assistant to write the following prompt:

- I want you to create 2 tables. I want them to be on customers and sales. Populate the tables with sample data into the public schema.
- Write a simple query to show these tables in the editor.

■ Selecting customer data

```
SELECT * FROM public.customer_profiles order by id limit 100;
```

■ Selecting Sales Orders

```
SELECT * FROM public.sales_orders order by id limit 100;
```

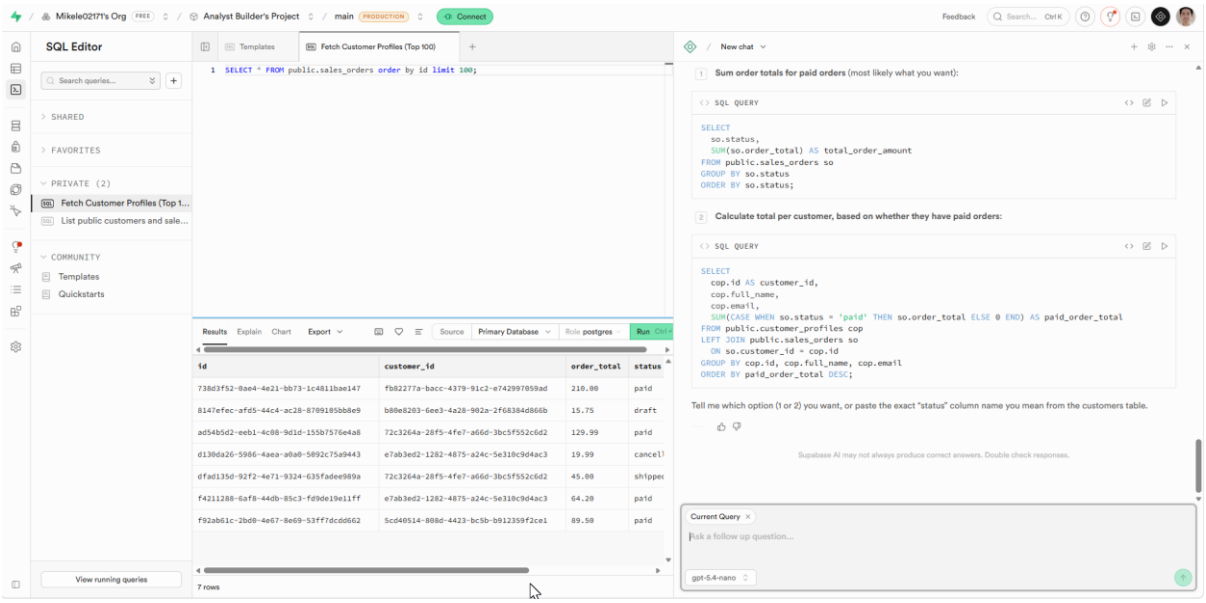
■ Check all paid orders

```
SELECT
 id,
 customer_id,
 order_total,
 status,
 created_at
FROM public.sales_orders
```

```
WHERE status = 'paid'
ORDER BY created_at DESC;
```

- calculate the order total based on status from the customers tables.

```
SELECT
  so.status,
  SUM(so.order_total) AS total_order_amount
FROM public.sales_orders so
GROUP BY so.status
ORDER BY so.status;
```



QUERY:

```
BEGIN;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Customers
CREATE TABLE IF NOT EXISTS
customers.customer_profiles (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 full_name text NOT NULL,
 email text NOT NULL UNIQUE,
 created_at timestamptz NOT NULL DEFAULT now()
);

-- Sales
CREATE TABLE IF NOT EXISTS sales.sales_orders (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 customer_id uuid NOT NULL REFERENCES
customers.customer_profiles(id) ON DELETE
CASCADE,
 order_total numeric(12,2) NOT NULL CHECK
(order_total >= 0),
 status text NOT NULL CHECK (status IN
('draft','paid','shipped','cancelled')),
 created_at timestamptz NOT NULL DEFAULT now()
);

-- Sample customers (insert-or-update by email)
```

```
WITH input_customers AS (
  SELECT * FROM (VALUES
    ('Ava Chen','ava.chen@example.com'),
    ('Noah Patel','noah.patel@example.com'),
    ('Sophia
Martinez','sophia.martinez@example.com'),
    ('Liam Brown','liam.brown@example.com'),
    ('Emma Johnson','emma.johnson@example.com')
  ) AS v(full_name,email)
)
INSERT INTO customers.customer_profiles
(full_name,email)
SELECT ic.full_name, ic.email
FROM input_customers ic
ON CONFLICT (email)
DO UPDATE SET full_name = EXCLUDED.full_name;

-- Sample orders
WITH input_orders AS (
  SELECT * FROM (VALUES
    ('ava.chen@example.com',   129.99::numeric,
'paid'    ::text),
    ('ava.chen@example.com',    45.00::numeric,
'shipped' ::text),
    ('noah.patel@example.com',  89.50::numeric,
'paid'    ::text),
```

```
   ('sophia.martinez@example.com', 210.00::numeric,
'paid'::text),
   ('liam.brown@example.com', 15.75::numeric,
'draft'::text),
   ('emma.johnson@example.com', 64.20::numeric,
'paid'::text),
   ('emma.johnson@example.com',  19.99::numeric,
'cancelled'::text)
  ) AS v(customer_email, order_total, status)
)
INSERT INTO sales.sales_orders (customer_id,
order_total, status)
SELECT cp.id, io.order_total, io.status
FROM input_orders io
JOIN customers.customer_profiles cp ON cp.email =
io.customer_email
ON CONFLICT DO NOTHING;

COMMIT;
```

Data Dictionaries, Overviews, and Calculations

Prompt when sending in the file:

- Tell me about this dataset and provide a detailed data directory.

NOTE: Better to run this in VSC, Jupiter or Databricks Free Edition Notebook to display these results

```
import pandas as pd

# Load dataset
file_path = "Tech Sales Dataset.csv"
df = pd.read_csv(file_path)

# Display basic information
print("Dataset Shape:")
print(df.shape)

print("
Column Names:")
print(df.columns.tolist())

print("
Dataset Info:")
print(df.info())
print("
First 5 Rows:")
print(df.head())

# Clean column names
df.columns = df.columns.str.strip()


# Convert date column
df['OrderDate'] = pd.to_datetime(df['OrderDate'])

# Clean currency columns
currency_cols = ['Price', 'Discount', 'Order Total']

for col in currency_cols:
```

```
df[col] = (
df[col]
.replace('[$,]', '', regex=True)
.astype(float)
)

# Generate summary statistics
print("
Summary Statistics:")
print(df.describe(include='all'))

# Create automated data dictionary
data_dictionary = pd.DataFrame({
'Column Name': df.columns,
'Data Type': df.dtypes.astype(str).values,
'Missing Values': df.isnull().sum().values,
'Unique Values': [df[col].nunique() for col in df.columns],
'Example Value': [df[col].dropna().iloc[0] for col in df.columns]
})

print("
Data Dictionary:")
print(data_dictionary)

# Save cleaned dataset
cleaned_file = 'cleaned_tech_sales_dataset.csv'
df.to_csv(cleaned_file, index=False)

# Save data dictionary
output_dictionary = 'tech_sales_data_dictionary.csv'
data_dictionary.to_csv(output_dictionary, index=False)

print(f"
Cleaned dataset saved as: {cleaned_file}")
print(f"Data dictionary saved as: {output_dictionary}")
```

- Go more in depth with the data cleaning suggestions. Rank them in order of priority. (see the pdf attachment, Tech Sales Dataset Overview and Data Dictionary)

- I want to know how many people got a discount on their order.

```
import pandas as pd

# Load dataset
df = pd.read_csv("Tech Sales Dataset.csv")

# Clean column names
df.columns = df.columns.str.strip()

# Convert Discount column to numeric
df['Discount'] = (
    df['Discount']
    .replace('[$,]', '', regex=True)
    .astype(float)
)

# Count discounted orders
discounted_orders = (df['Discount'] > 0).sum()

# Total orders
total_orders = len(df)

# Percentage
discount_percentage = (
    discounted_orders / total_orders
) * 100

print("Discounted Orders:", discounted_orders)
print("Percentage:", round(discount_percentage, 2), "%")
```

Answer:

Discounted Orders: 2833

Percentage: 56.94 %

Brainstorming and Generating Ideas

- I'm a new data analyst at this company that sells things. I was handled this dataset and they said to analyze it. I don't know what that means. Can you help generate ideas on how we can analyze this that would be helpful to the business.

NOTE: Answers may differ from the video, you may have to be specific with your prompts.

- For operational questions: Large Orders: Which customers/products bring in bulk sales? Are there "low-value" customers (lots of small orders)? Are certain product categories only appealing to Retail vs Wholesale? (What are different things we can analyze in this area?)
- For Bulk Orders and Efficiency, which do you think would the most helpful to the company.
- Let's break down this down into the actual code and insights. Provide documentation and visualizations if you can. (Refer to Bulk Orders Operational Efficiency Analysis Python.pdf for more information)

What AI can and Can't Do

What AI CAN Do:

- Recognize patterns in data and make predictions
- Assist with coding – like data cleaning and transforming data
- Provide assistance with decision-making and gathering insights
- Brainstorming ideas

What AI CANNOT Do:

- Truly understand meaning the way humans do
- Guarantee factually correct answers every time (it can hallucinate)

- Make moral or ethical judgments

Ethics of Using AI

- AI can reflect or amplify biases in the data; always validate results for fairness.
- Protect sensitive information and avoid sharing confidential data with AI tools.
- Treat AI as a supportive tool, but take responsibility for final decisions and outcomes.